```
# Data handling libraries
import pandas as pd # Used for data manipulation and analysis, especially with DataFrames
import numpy as np # Used for numerical operations, especially with arrays

# Visualization libraries
import matplotlib.pyplot as plt # Used for creating static, interactive, and animated visualizations
import seaborn as sns # Built on Matplotlib, provides a high-level interface for drawing attractive statistical graphics

# Dataset library from Hugging Face
from datasets import load_dataset # Used to easily load and share datasets from the Hugging Face Hub

# Transformer libraries
from transformers import BertTokenizer, BertForSequenceClassification, Trainer, TrainingArguments # Essential components fr

# Evaluation metrics
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, confusion_matrix # Used to evaluate the perfor
```

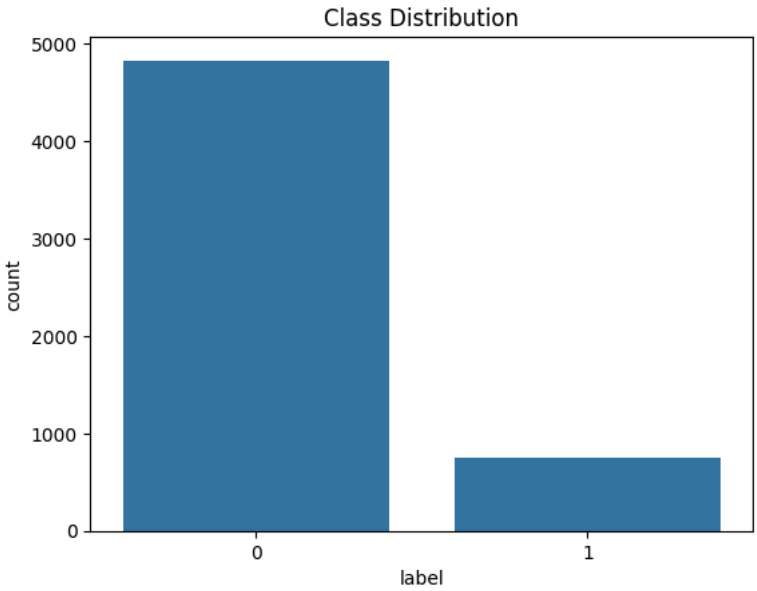
```
# Load SMS spam dataset from Hugging Face Datasets Hub
dataset = load_dataset("sms_spam")

# Convert the 'train' split of the dataset to a pandas DataFrame for easier inspection and manipulation
df = dataset["train"].to_pandas()

# Display the first 5 rows of the DataFrame to inspect the data structure and content
print(df.head())
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
README.md:        4.98k/?  [00:00<00:00,  95.2kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
plain_text/train-00000-of-00001.parquet:  100%                    359k/359k  [00:00<00:00,  563kB/s]
Generating train split:  100%                    5574/5574  [00:00<00:00,  6360.25  examples/s]
                                                 sms  label
0  Go until jurong point, crazy.. Available only ...      0
1                      Ok lar... Joking wif u oni...\n      0
2  Free entry in 2 a wkly comp to win FA Cup fina...      1
3  U dun say so early hor... U c already then say...      0
4  Nah I don't think he goes to usf, he lives aro...      0
```

```
# Print the names of all columns in the DataFrame to understand the available features
print(df.columns)
```

```
Index(['sms', 'label'], dtype='object')
```

```
# Create a countplot to visualize the distribution of the 'label' column (0 for ham, 1 for spam)
sns.countplot(x=df['label'])
# Set the title of the plot for clarity
plt.title("Class Distribution")
# Display the plot
plt.show()
```




```
# Initialize the BERT tokenizer for the 'bert-base-uncased' model.
# This tokenizer converts text into numerical tokens that the BERT model can understand.
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
```

```
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 765B/s]
vocab.txt: 100%  232k/232k [00:00<00:00, 5.62MB/s]
tokenizer.json: 100%  466k/466k [00:00<00:00, 8.68MB/s]
```

```
def tokenize(example):
    # Tokenize the 'sms' text, adding padding, truncation, and setting a max_length.
    # This prepares the text data into a format suitable for BERT.
    return tokenizer(example["sms"], padding="max_length", truncation=True, max_length=128)

# Apply the 'tokenize' function to the entire dataset using the .map() method.
# 'batched=True' processes multiple examples at once for efficiency.
tokenized_dataset = dataset.map(tokenize, batched=True)
```

```
Map: 100%  5574/5574 [00:02<00:00, 2351.49 examples/s]
```

```
# Rename the 'label' column to 'labels' to match the expected input name for Hugging Face Trainer API.
tokenized_dataset = tokenized_dataset.rename_column("label", "labels")
# Set the format of the dataset to 'torch' tensors and specify the columns to be returned ('input_ids', 'attention_mask', 'l
# This prepares the dataset for use with PyTorch models and the Trainer.
tokenized_dataset.set_format("torch", columns=["input_ids", "attention_mask", "labels"])
```

```
# Split the 'train' portion of the tokenized dataset into training and testing sets.
# 'test_size=0.2' means 20% of the data will be used for testing, and 80% for training.
split = tokenized_dataset["train"].train_test_split(test_size=0.2)

# Assign the training split to 'train_dataset'
train_dataset = split["train"]
# Assign the testing split to 'test_dataset'
test_dataset = split["test"]
```

```
# Load a pre-trained BERT model for sequence classification.
# 'bert-base-uncased' specifies the base BERT model without a cased vocabulary.
# 'num_labels=2' indicates that the model will classify into 2 categories (e.g., spam/ham).
model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=2)
```

```
config.json: 100%                                                    570/570 [00:00<00:00, 52.0kB/s]

model.safetensors: 100%                                                  440M/440M [00:02<00:00, 288MB/s]

Loading weights: 100%                                                 199/199 [00:00<00:00, 645.97it/s, Materializing param=bert.pooler.dense.weight]
BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                         | Status     |
-------------------------------------------+------------+-
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |
cls.predictions.bias                       | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.seq_relationship.weight                | UNEXPECTED |
cls.predictions.transform.dense.bias       | UNEXPECTED |
cls.predictions.transform.dense.weight     | UNEXPECTED |
cls.seq_relationship.bias                  | UNEXPECTED |
classifier.bias                            | MISSING    |
classifier.weight                          | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream t
```

```
# Update the transformers library to its latest version.
# This ensures access to the newest features, bug fixes, and API changes.
!pip install -U transformers
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.2.0)
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (1
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (26.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2025.11.3)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from transformers) (0.24.0)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.3)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transf
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0
Requirement already satisfied: shellingham in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->tra
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0
Requirement already satisfied: typer>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from typer-slim->transformers) (0.2
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->h
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim->tran
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim->tran
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-sl
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer>=0
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer>
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3
```

```
# Defines training parameters for model training process
training_args = TrainingArguments(

    output_dir="./results",          # Directory where training outputs will be saved

    learning_rate=2e-5,              # Controls how fast the model learns

    per_device_train_batch_size=8,   # Number of samples processed at once during training

    per_device_eval_batch_size=8,    # Batch size used during evaluation

    num_train_epochs=3               # Number of times the model sees entire dataset
)
```

```
# This function calculates model performance metrics
# It is used by Trainer during evaluation phase
def compute_metrics(pred):

    # Extracts actual labels from predictions
    labels = pred.label_ids

    # Converts model output probabilities into predicted class labels
    preds = pred.predictions.argmax(-1)

    # Calculates precision, recall and F1 score
    precision, recall, f1, _ = precision_recall_fscore_support(labels, preds, average='binary')

    # Calculates overall accuracy of predictions
```

```
    acc = accuracy_score(labels, preds)

    # Returns all evaluation metrics as dictionary
    return {
        "accuracy": acc,
        "precision": precision,
        "recall": recall,
        "f1": f1,
    }
```

```
# Creates Trainer object to handle training and evaluation automatically
trainer = Trainer(

    model=model,                      # The BERT classification model

    args=training_args,               # Training configuration parameters

    train_dataset=train_dataset,      # Dataset used for training

    eval_dataset=test_dataset,        # Dataset used for evaluation

    compute_metrics=compute_metrics  # Function to calculate evaluation metrics
)
```